

Блок 1: Вопросы по программированию и архитектуре

1. Почему выбрана модульная структура, а не единый скрипт?

Ответ: Модульная структура обеспечивает:

- **Разделение ответственности** — каждый метод интерполяции изолирован в своём файле, что упрощает тестирование и отладку
- **Расширяемость** — добавление нового метода интерполяции сводится к созданию нового файла в пакете `interpolation/` без изменения существующего кода
- **Переносимость** — метод прогонки из `tridiagonal.py` может использоваться не только кубическим сплайном, но и любым другим модулем
- **Соответствие принципу единственной ответственности (SRP)** — каждый модуль решает ровно одну задачу

2. Почему для полинома Лагранжа выбрана барицентрическая форма?

Ответ: Каноническая форма полинома Лагранжа:

$$P(x) = \sum_{j=0}^n y_j \prod_{k \neq j} \frac{x - x_k}{x_j - x_k}$$

страдает от двух проблем:

- **Вычислительная сложность $O(n^2)$** для каждой точки
- **Численная неустойчивость** при большом числе узлов из-за многократного перемножения чисел с плавающей точкой

Барицентрическая форма:

$$P(x) = \frac{\sum_{j=0}^n \frac{y_j}{x - x_j} b_j}{\sum_{j=0}^n \frac{b_j}{x - x_j}}$$

где веса b_j вычисляются один раз за $O(n^2)$, а затем каждое значение — за $O(n)$. Это **устойчивее** и **быстрее** при многократных вычислениях.

3. В чём преимущество полинома Ньютона перед Лагранжем?

Ответ: Кубический сплайн строится на основе второй производной (момента) M_i . Система уравнений для моментов имеет размерность $(n - 2) \times (n - 2)$ для внутренних узлов (с учётом граничных условий $M_0 = M_{n-1} = 0$). При $n < 3$ матрица системы пуста или имеет размерность 0, что не позволяет однозначно определить сплайн. В коде это проверяется явно:

При этом SciPy может использоваться для **верификации** результатов (сравнение с эталоном).

4. Почему метод прогонки написан вручную, а не взят из SciPy?

Ответ: Тому две причины:

- **Научная:** продемонстрировать понимание внутреннего устройства алгоритма. Метод прогонки (алгоритм Томаса) — классический метод решения тридиагональных СЛАУ за $O(n)$, и его ручная реализация показывает квалификацию в области численных методов
- **Юридическая:** для регистрации программы для ЭВМ важно, чтобы ключевые алгоритмы были оригинальной разработкой, а не заимствованием из сторонних библиотек.

При этом SciPy может использоваться для **верификации** результатов (сравнение с эталоном).

5. Как обрабатываются граничные случаи (x совпадает с узлом)?

Ответ: В методе `_call__` класса `LagrangePolynomial` явно проверяется совпадение:

Это важно, потому что барицентрическая формула содержит деление на $(x - x_j)$, и при точном совпадении возникает деление на ноль. Порог `1e-14` выбран как компромисс между машинной точностью double и возможной погрешностью округления.

6. Какой порядок сложности у реализованных алгоритмов?

Ответ: [Алгоритм] [Построение | Вычисление в одной точке | Вычисление в m точках] [-----] [Лагранж (веса) | $O(n^2)$ | $O(n)$ | $O(m \cdot n)$] [Ньютон (таблица) | $O(n^2)$ | $O(n)$ | $O(m \cdot n)$] [Линейный сплайн | $O(n)$ | $O(\log n)$ | $O(m \log n)$] [Кубический сплайн | $O(n)$ | $O(\log n)$ | $O(m \log n)$]

Для сплайнов используется `np.searchsorted` — бинарный поиск по отсортированному массиву узлов за $O(\log n)$.

7. Почему кубический сплайн требует минимум 3 узлов?

Ответ: Кубический сплайн строится на основе второй производной (момента) M_i . Система уравнений для моментов имеет размерность $(n - 2) \times (n - 2)$ для внутренних узлов (с учётом граничных условий $M_0 = M_{n-1} = 0$). При $n < 3$ матрица системы пуста или имеет размерность 0, что не позволяет однозначно определить сплайн. В коде это проверяется явно:

При этом SciPy может использоваться для **верификации** результатов (сравнение с эталоном).

8. Как устроена обработка ошибок в программе?

Ответ: Используется несколько уровней:

- **Проверка входных данных** — соответствие размерностей массивов узлов и значений, строгое возрастание узлов для сплайнов
- **Проверка вырожденности матрицы** — в методе прогонки проверяется, что диагональный элемент не близок к нулю (порог `1e-15`), иначе выбрасывается `ValueError`
- **Информирование пользователя** — при отсутствии LaTeX программа не падает, а выводит понятную инструкцию и создаёт HTML-отчёт

9. Почему для визуализации используется matplotlib.use('Agg')?

Ответ: `Agg` (Anti-Grain Geometry) — это **неинтерактивный бэкэнд**, который позволяет генерировать графики без графического интерфейса (GUI). Это необходимо для:

- Запуска на серверах без дисплея (SSH-доступ)
- Пакетной генерации графиков в консольном режиме
- Избегания ошибок вида `TypeError: no display name`

В Jupyter Notebook бэкэнд переопределяется автоматически на интерактивный через `matplotlib inline`.

10. Как обеспечивается воспроизводимость результатов?

Ответ:

- **Детерминированные алгоритмы** — все методы интерполяции чисто детерминированы, без случайных компонентов
- **Фиксированные тестовые функции** — аналитически заданные, без экспериментальных данных
- **Сохранение параметров** — в отчёте фиксируются все параметры запуска (функция, число узлов, тип сетки)
- **Сохранение сырых данных** — результаты экспортируются в CSV для возможности повторного анализа

Блок 2: Вопросы по вычислительной математике

11. Что такое феномен Рунге и как он проявляется в программе?

Ответ: Феномен Рунге — это **расходимость полиномиальной интерполяции высокого порядка на равномерной сетке** для некоторых функций, даже аналитических на всём отрезке. Классический пример — функция Рунге:

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1]$$

При увеличении числа узлов n максимальная погрешность **растёт**, а не убывает. В программе это демонстрируется визуально: осцилляции полинома у краёв отрезка увеличиваются с ростом n .

12. Почему сетка Чебышёва помогает бороться с феноменом Рунге?

Ответ: Узлы Чебышёва сгущаются к краям отрезка пропорционально:

$$x_k = \cos\left(\frac{(2k - 1)\pi}{2n}\right)$$

Это соответствует **минимизации константы Лебега** — множителя, связывающего погрешность интерполяции с погрешностью наилучшего приближения. Константа Лебега для равномерной сетки растёт как $O(2^n/n)$, а для чебышёвской — как $O(\ln n)$. Меньшая константа Лебега означает меньшие осцилляции.

13. В чём разница между полиномами Лагранжа и Ньютона математически?

Ответ: Математически это **один и тот же полином** — интерполяционный полином степени $n - 1$, проходящий через n точек, единственен. Различаются только **формы записи**:

- **Лагранж:** $P(x) = \sum y_j \cdot \ell_j(x)$ (где ℓ_j — базисные полиномы)
- **Ньютон:** $P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$

Численная разница (в пределах машинной точности) подтверждается в ячейке 4 блокнота.

14. Какие граничные условия используются для кубического сплайна и почему?

Ответ: Используются **естественные граничные условия**: $S''(a) = S''(b) = 0$. Это означает, что моменты $M_0 = M_{n-1} = 0$.

Выбор обоснован:

- Это простейший и наиболее распространённый вариант
- Соответствует физической аналогии с упругой линейкой (гибкой рейкой), которая принимает форму с нулевой кривизной на свободных концах
- Минимизирует интеграл от квадрата второй производной — сплайн обладает свойством минимальной кривизны

Другие варианты (закреплённые концы, циклические условия) могут быть добавлены как расширение.

15. Как вычисляется среднеквадратичная норма погрешности?

Ответ: Используется **составная формула трапеций** на мелкой сетке из 1000 точек:

$$\|f - P\|_2 = \sqrt{\int_a^b (f(x) - P(x))^2 dx} \approx \sqrt{h \left[\frac{(f_0 - P_0)^2}{2} + \sum_{i=1}^{n-2} (f_i - P_i)^2 + \frac{(f_{n-1} - P_{n-1})^2}{2} \right]}$$

где $h = (b - a)/(n - 1)$. Формула трапеций имеет второй порядок точности по h , что достаточно для практической оценки погрешности интерполяции.

16. Почему равномерная норма оценивается на дискретной сетке, а не ищется аналитический максимум?

Ответ: Аналитический поиск максимума погрешности $|f(x) - P(x)|$ на отрезке требует решения уравнения $f'(x) - P'(x) = 0$, что для полиномов высокой степени сводится к поиску корней многочлена — задаче не проще исходной. Оценка на мелкой сетке из 1000 точек даёт практически точное значение максимума для гладких функций, а для гарантии используется более плотная сетка вблизи ожидаемых экстремумов.

17. Почему кубический сплайн даёт лучшие результаты для функции |x|?

Ответ: Функция $|x|$ имеет **излом** в точке $x = 0$ — разрыв первой производной. Глобальный полином пытается приблизить её одной гладкой кривой, что приводит к **осцилляциям Гиббса** — волнообразным отклонениям вблизи излома, не исчезающим с ростом степени полинома. Сплайн же является кусочно-полиномиальным и может иметь изломы второй производной в узлах, что позволяет ему лучше адаптироваться к негладкому поведению функции.

18. Чем опасна каноническая форма полинома через СЛАУ Вандермонда?

Ответ: Матрица Вандермонда $V_{ij} = x_i^j$ имеет **число обусловленности**, экспоненциально растущее с размером (особенно на равномерной сетке). Уже при $n \approx 15$ число обусловленности достигает 10^{12} , что приводит к полной потере точности при решении СЛАУ в double precision. Именно поэтому этому метод не включён в программу, а вместо него используются устойчивые формы Лагранжа и Ньютона.

19. Как ведёт себя погрешность линейного сплайна при увеличении числа узлов?

Ответ: Для линейного сплайна справедлива оценка:

$$\|f - S\|_{\infty} \leq \frac{h^2}{8} \max_{x \in [a,b]} |f''(x)|$$

где $h = \max(x_{i+1} - x_i)$. Погрешность убывает как **$O(h^2)$** — второй порядок сходимости. На графиках в логарифмическом масштабе это выглядит как прямая с наклоном -2 . Это медленнее, чем у кубического сплайна ($O(h^4)$), но линейный сплайн проще и не требует решения СЛАУ.

20. Что произойдёт, если подать на вход неотсортированные узлы?

Ответ: Для полиномов Лагранжа и Ньютона порядок узлов **не имеет значения** математически (полином степени $n - 1$ через n точек единственен при любом порядке). Однако для сплайнов порядок критичен: они требуют строго возрастающих узлов, что проверяется явно:

Это связано с тем, что сплайн определяется как кусочная функция на отрезках $[x_i, x_{i+1}]$, и для корректного поиска отрезка через `searchsorted` требуется упорядоченность.

Блок 3: Общие вопросы по программе и регистрации

21. В чём новизна программы по сравнению с аналогами?

Ответ:

- Все методы **реализованы с нуля** без использования готовых библиотек интерполяции — это educational / research tool
- Проводится **комплексное сравнение** двух классов методов (глобальные полиномы vs сплайны) на едином наборе тестовых функций
- Автоматическая генерация научного отчёта (LaTeX — PDF) с таблицами и графиками — готовый документ для публикации
- Интерактивный Jupyter Notebook для исследовательской работы

22. Какие зависимости используются и почему именно они?

Ответ: Минимальный набор:

- **Numpy** — работа с массивами, векторизация вычислений. Стандарт де-факто в научном Python
- **Matplotlib** — визуализация. Позволяет сохранять графики в векторном PDF для LaTeX
- **Jupyter** (опционально) — интерактивный анализ

Зависимости от SciPy **намеренно исключены** для расчётной части, чтобы продемонстрировать самостоятельную реализацию алгоритмов.

23. Как программа обрабатывает ситуацию отсутствия LaTeX?

Ответ: В модуле `report_generator.py` метод `compile_pdf()` перехватывает `FileNotFoundError` при попытке запустить `pdflatex`. Программа:

1. Выводит подробную инструкцию по установке LaTeX (MiKTeX, TeX Live, MacTeX)
2. Предлагает ссылку на онлайн-компилятор OnlineLaTeX
3. Создаёт **HTML-отчёт** с MathJax — полноценную замену PDF для просмотра в браузере
4. Сохраняет `.tex`-файл для ручной компиляции

Никакого аварийного завершения не происходит.

24. Как подтверждается корректность реализации алгоритмов?

Ответ: Несколько способов верификации:

- **Внутренняя проверка:** полиномы Лагранжа и Ньютона должны давать одинаковый результат (проверяется в блокноте, расхождение $\sim 10^{-16}$)
- **Проверка в узлах:** интерполюнт должен точно проходить через узлы (свойство интерполяции)
- **Асимптотика:** погрешность должна убывать с ростом числа узлов для гладких функций (проверяется на `sin(x)`)
- **Сравнение с эталоном:** SciPy может использоваться для независимой проверки (не встроено, но легко добавить)

25. Какие форматы выходных данных поддерживаются?

Ответ:

- **График:** PDF (векторный, для LaTeX) и PNG (растровый, для быстрого просмотра)
- **Таблицы:** CSV (для анализа в Excel/Python/pandas)
- **Отчёт:** LaTeX → PDF, HTML с MathJax
- **Лог:** текстовый вывод в консоль с численными значениями погрешностей

26. Как масштабируется программа на большое число узлов?

Ответ: Основные ограничения:

- **Полиномы Лагранжа/Ньютона:** $O(n^2)$ по памяти для весов/разностей и $O(n^2)$ по времени для построения. При $n = 1000$ время построения ~ 1 секунда
- **Сплайны:** $O(n)$ по времени и памяти. Могут работать с десятками тысяч узлов
- **Визуализация:** всегда используется фиксированная сетка из 1000 точек для графиков, независимо от числа узлов интерполяции

27. Какие краевые условия кубического сплайна можно добавить?

Ответ: Текущая реализация использует естественные условия ($M_0 = M_{n-1} = 0$). Можно добавить:

- **Закреплённые концы** — заданы первые производные $S'(a)$ и $S'(b)$
- **Периодические условия** — $S'(a) = S'(b)$, $S''(a) = S''(b)$ (для периодических функций)
- **Условия «лот-а-клот»** — непрерывность третьей производной на первом и предпоследнем узлах (используется в MATLAB по умолчанию)

Все они сводятся к модификации первой и последней строк тридиагональной системы, сохраняя метод прогонки.

28. Как организован выбор тестовых функций?

Ответ: Подобраны четыре функции, покрывающие разные классы сложности:

- $\sin(x)$ — **гладкая периодическая** (лёгкий тест)
- $\frac{1}{1+25x^2}$ — **функция Рунге** (тест на феномен Рунге)
- $|x|$ — **негладкая с изломом** (тест на явление Гиббса)
- $e^{-x} \cdot \sin(5x)$ — **осциллирующая с затуханием** (тест на разрешающую способность)

Набор расширяем — достаточно добавить запись в словарь `get_test_functions()`.

29. Почему выбраны именно Python и Jupyter, а не компилируемый язык?

Ответ:

- **Python** — стандарт в научных вычислениях, огромная экосистема (Numpy, Matplotlib, pandas)
- **Jupyter** — де-факто стандарт для воспроизводимых исследований, позволяет совмещать код, формулы и графики
- **Прототипирование** — Python позволяет быстро реализовать и проверить алгоритмы
- **Доступность** — бесплатная, кроссплатформенная, не требует компиляции

Для production-версии ядро алгоритмов может быть переписано на C++/Cython, но для научной демонстрации Python оптимален.

30. Как можно расширить программу в будущем?

Ответ:

- Добавить **сплайны Эрмита** (с заданными производными)
- Реализовать **В-сплайны** — альтернативный базис для сплайнов
- Добавить **многомерную интерполяцию** (билинейную, бикубическую)
- Реализовать **адаптивный выбор узлов** по величине погрешности
- Интегрировать сравнение с **нейросетевыми аппроксимациями**
- Создать **графический интерфейс** (PyQt/MinQt)
- Оптимизировать через **Cython/Numba** для работы с большими данными

Архитектура модульной структуры позволяет добавлять новые методы без изменения существующего кода.

Важно!!!!

Вопрос про интерполяцию по Ньютону и по Лагранжу!!!

Почему методы Лагранжа и Ньютона дают одинаковые результаты

1. Интерполяционный полином степени $n - 1$, проходящий через n заданных точек, математически **единственен** — это фундаментальная теорема теории интерполяции, не зависящая от способа построения полинома.

2. Полиномы Лагранжа и Ньютона — это не разные полиномы, а лишь **две разные формы записи** одного и того же многочлена, подобно тому как число можно записать в десятичной или двоичной системе.

3. Форма Лагранжа представляет полином как сумму базисных полиномов $\ell_j(x)$, каждый из которых равен 1 в «своем» узле и 0 во всех остальных, а форма Ньютона — как сумму разделённых разностей, умноженных на накопительные произведения $(x - x_0)(x - x_1) \dots$

4. Разница между ними — исключительно **вычислительная**: алгоритм Лагранжа требует $O(n^2)$ операций для добавления нового узла, тогда как алгоритм Ньютона позволяет добавит узел за $O(n)$ без полного пересчёта.

5. При вычислении значений в одних и тех же точках оба метода дают **численно идентичные результаты** с точностью до погрешностей машинной арифметики (порядка 10^{-14} – 10^{-16} для double precision).

6. На графиках линия Лагранжа **присутствует всегда**, но она полностью совпадает с линией Ньютона, поэтому визуально видна только одна из них — это нормальное поведение, подтверждающее корректность обеих реализаций.

7. Проверить, что оба метода действительно работают, можно в Jupyter Notebook (Часть 4), где явно строится график разности $|P_{\text{Лагранж}}(x) - P_{\text{Ньютон}}(x)|$ — её значение не превышает 10^{-14} на всём отрезке.